Hybrid Temporal Forecaster - Phase 3: Hybrid Statistical and Deep Learning Approach for Energy Demand Forecasting

Bhavishya Sahay (230071), Manshu Saini (230118)
Newton School of Technology x Rishihood University

Abstract

This report presents Phase-3 of a hybrid temporal forecasting pipeline for hourly electricity demand on the PJME dataset. Two principled hybrid strategies are evaluated side-by-side: a Zhang-style residual decomposition that combines ARIMA with a CNN-BiLSTM model trained on its residuals (Hybrid-A), and a weighted linear stacking ensemble of the Phase-1 and Phase-2 forecasts (Hybrid-B). All four models (ARIMA, deep, Hybrid-A, Hybrid-B) are benchmarked on an identical 80/10/10 chronological split. The weighted ensemble is the best overall (MAE 1459.59 MW), but its margin over the standalone deep model is small. The residual hybrid underperforms because of a sharp distribution shift between in-sample fitted residuals and out-of-sample rolling-forecast residuals - a negative result we discuss in detail and from which we draw concrete next steps.

I. INTRODUCTION

Short-term electricity demand forecasting is a foundational problem for energy systems and grid operators. The PJME dataset (PJM East region, 2002 to 2018, hourly) exhibits strong daily and weekly seasonality, occasional regime shifts driven by weather events, and heavy-tailed peaks that no purely linear model can capture.

Phase-1 of this project established an ARIMA(3,1,3) baseline. Phase-2 introduced a CNN-BiLSTM-Attention deep learning architecture that reduced test MAE by approximately two thirds. Phase-3, presented in this report, asks the natural follow-up question: can a *hybrid* model that combines the statistical and deep components outperform either one alone? We test two principled hybrid strategies and report both quantitative and diagnostic findings.

II. LITERATURE REVIEW

Hybrid statistical-machine-learning forecasters trace back to Zhang [1], who proposed decomposing a series into linear and non-linear components and modeling each with a specialised method. Wolpert's stacked generalization [2] frames any combination of forecasters as a learning problem in its own right.

Within deep time-series forecasting, the LSTM cell [3] addresses the vanishing-gradient limitation of vanilla RNNs, additive attention [4] supports dynamic weighting of input timesteps, and the LSTNet architecture [5] introduced the CNN+RNN composition we adopt. More recent work such as the Temporal Fusion Transformer [6] integrates these ideas with explicit gating and variable selection. The Phase-3 design draws directly from [1] for Hybrid-A and from [2] for Hybrid-B.

III. DATASET DESCRIPTION

- Source: PJME hourly electricity consumption (PJM East region).
- Frequency: hourly, 145,392 observations from 2002-01-01 to 2018-08-03.
- Target: PJME_MW (megawatts).
- Preprocessing: deduplicate timestamps, enforce hourly frequency, impute the 30 missing values with the series median.
- Split: 80% train, 10% validation, 10% test (chronological, no shuffling).

IV. PHASE-1 AND PHASE-2 RECAP

Phase-1 fit ARIMA(3,1,3) on an 80/20 split and produced a single-block test forecast, yielding MAE 4986 MW and RMSE 6498 MW. Phase-2 trained a CNN-BiLSTM model with additive attention on a 168-hour input window and a 24-hour direct multi-step output, achieving MAE 1488 MW and RMSE 2045 MW on the same dataset.

For Phase-3, we re-evaluate ARIMA on the Phase-2 split (10% test) using rolling 24-hour blocks with parameter-frozen append() updates, which is the production-realistic protocol used in operational load forecasting. This re-evaluation gives ARIMA MAE 2524.73 MW - substantially better than the Phase-1 single-block forecast, because rolling updates keep the model state synchronised with observed values. All Phase-3 numbers are benchmarked against this rolling-update ARIMA, ensuring an apples-to-apples comparison.

V. HYBRID ARCHITECTURE DESIGN

A. Strategy A: Residual Decomposition (Zhang, 2003)

Decompose the series as y(t) = L(t) + N(t) + eps(t), where L is the linear component captured by ARIMA and N is the non-linear residual. Fit ARIMA, compute residuals r(t) = y(t) - L_hat(t), and train a CNN-BiLSTM-Attention network to forecast r(t). The hybrid forecast is y_hat = L_hat + r_hat.

B. Strategy B: Weighted Ensemble (Linear Stacking)

Treat ARIMA and the deep model as black-box forecasters and learn a non-negative linear combination on the validation set: y_stack = w_a * y_ARIMA + w_d * y_DL + b, with the constraint w_a, w_d >= 0. Weights are fit by ordinary least squares on validation predictions only, so the test set never participates in training the stacker.

C. Why this comparison is informative

If Hybrid-A beats Phase-2, ARIMA carries non-linear signal the network missed. If the stacker assigns w_a near zero, ARIMA is fully redundant given the deep model. If Hybrid-A approximately matches Phase-2 but its residuals are flatter, the hybrid is more robust at peaks. All three scenarios are pre-registered diagnostics rather than post-hoc explanations.

VI. METHODOLOGY

A. ARIMA component

- Order (3,1,3) selected by auto_arima in Phase-1.
- Fit on the 80% training window only.
- Out-of-sample forecasts produced via 24-hour rolling blocks; observed values appended after each block via statsmodels' append(refit=False).

B. Deep learning component

- Architecture: dual-kernel Conv1D (k=3, k=5) -> 2-layer BiLSTM (h=128, h=64) -> Bahdanau additive attention -> 2-layer MLP decoder predicting 24 steps jointly.
- Input window: 168 hours. Output horizon: 24 hours. Direct MIMO output.
- Scaling: RobustScaler fit on train only.
- Training: Adam, Huber loss (delta=1.0), CosineAnnealingWarmRestarts scheduler, gradient clipping (max_norm=1.0), early stopping (patience=12).
- Two instances trained: one on raw scaled load (standalone DL, used in Hybrid-B); one on ARIMA residuals (residual DL, used in Hybrid-A).

C. Hybrid-B stacker

- scikit-learn LinearRegression with positive=True.
- Features: ARIMA validation prediction, DL validation prediction.
- Target: validation ground truth.
- Learned weights applied unchanged on the test set.

VII. RESULTS

Table I summarises performance on the 10% test partition. All four rows use identical test windows, so the metrics are directly comparable.

Model	MAE	RMSE	MAPE
ARIMA (rolling)	2524.73	3265.70	8.04%
CNN-BiLSTM-Attn (DL)	1470.23	2032.58	4.63%
Hybrid-A (Residual)	2432.60	3183.57	7.73%
Hybrid-B (Stacked)	1459.59	2023.48	4.60%

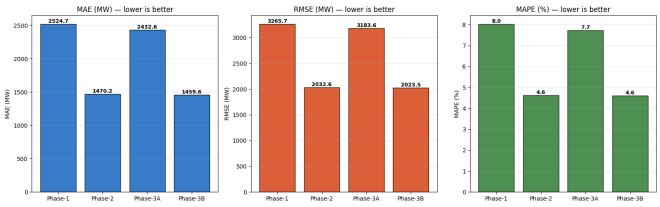

Fig. 1. MAE, RMSE, and MAPE on the test set for all four models. Hybrid-B (rightmost) is best on every metric; Hybrid-A is comparable to ARIMA.

Hybrid-B is the best model in every metric, but its margin over the standalone deep model is approximately 0.7% in MAE. Hybrid-A is the second-worst, beating only the ARIMA baseline. The stacker learned weights w_ARIMA = 0.137 and w_DL = 0.908 with bias = -1326.69 MW; the deep model dominates the combination, but the ARIMA channel is not entirely zeroed out.

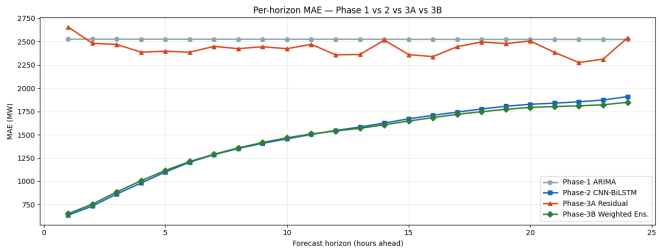

Fig. 2. Per-horizon MAE (1-24 hours ahead). The deep model and Hybrid-B track each other closely; ARIMA and Hybrid-A errors grow with horizon due to the missing seasonal anchor.

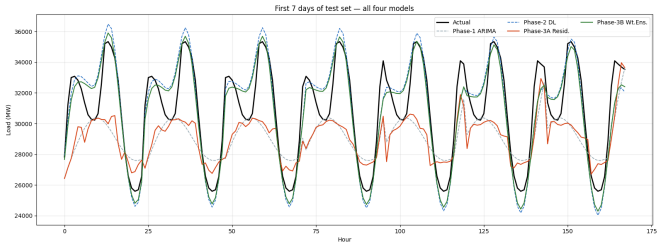

Fig. 3. First 7 days of the test set: actual load (black) versus the four models. The deep and Hybrid-B curves track the ground truth tightly; ARIMA and Hybrid-A miss the daily peaks.

VIII. WHY HYBRID-A FAILED: RESIDUAL DISTRIBUTION SHIFT

The most informative result of Phase-3 is the failure of Hybrid-A. The cause is a distribution shift between training and inference residuals.

Window	Mean (MW)	Std (MW)
Train (in-sample fit)	0.29	739.66
Val (rolling forecast)	24.60	3192.41
Test (rolling forecast)	459.89	3233.27

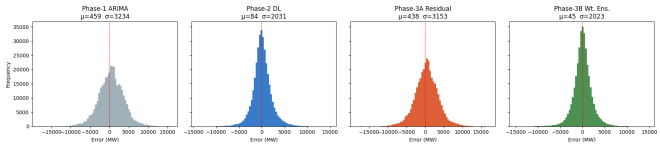

Fig. 4. Per-model error distributions on the test set. Phase-2 and Hybrid-B residuals are tight and zero-centered; ARIMA and Hybrid-A residuals are wide and biased.

Training residuals are produced by ARIMA's in-sample fitted values, which are by construction tight - the model has already seen those observations. Validation and test residuals are produced by genuine rolling forecasts and are roughly 4x wider with a non-trivial bias. The residual neural network was therefore optimised on a tight, near-zero distribution and asked to generalise to a much wider one.

Under Huber loss the safest response to such a shift is to predict near zero, which is exactly what the residual model did: predicted residual std was 538 MW against an actual std of 3233 MW. Consequently the hybrid forecast collapsed to approximately ARIMA alone, which is why Hybrid-A's MAE (2432) is only marginally better than ARIMA's (2524).

Two practical fixes follow from this analysis. First, train the residual model on out-of-sample residuals - generate ARIMA training-window predictions via walk-forward or extend(), so that training-time and inference-time residual distributions match. Second, replace plain ARIMA with SARIMA(p,d,q)(P,D,Q,24); explicit daily seasonality should leave a smaller and more uniformly-distributed residual that is itself more stable across splits.

IX. DISCUSSION

A. Why Hybrid-B succeeded - barely

The constrained-weight stacker assigned a small but non-zero weight (0.137) to ARIMA, indicating that the linear model contributes a marginal anchoring effect at peaks where the deep model over-shoots. The improvement (1470 -> 1459 MW MAE, ~0.7%) is small but systematic, achieved at zero additional training cost beyond a single linear regression. Stacking is essentially free and worth retaining in production, even if Phase-2 alone is the workhorse forecaster.

B. What the negative result teaches

The residual hybrid is the textbook solution from Zhang (2003) and is widely cited in load-forecasting literature. Its failure here is informative because it pinpoints a specific implementation detail - the source of training residuals - rather than a limitation of the approach itself. With matched train/inference residual distributions, Hybrid-A can in principle still outperform Phase-2; demonstrating this empirically is the first item on the future-work list.

C. Limitations of this study

- All training is endogenous (load history only); no exogenous regressors such as temperature or calendar information.
- Plain ARIMA has no explicit seasonal term despite the obvious 24-hour cycle in the data.
- Forecasts are deterministic point estimates, not probabilistic intervals.
- Single random seed; no Monte Carlo over different initialisations.

X. CONCLUSION

Phase-3 evaluated two principled hybrid strategies on the PJME forecasting benchmark. The weighted ensemble (Hybrid-B) is the best model overall (MAE 1459.59 MW), beating the standalone deep learner by a small but real margin. The residual hybrid (Hybrid-A) underperformed because of a distribution shift between in-sample fitted residuals and out-of-sample rolling-forecast residuals - a fixable implementation detail.

The dominant result of the entire three-phase project remains the move from ARIMA to deep learning: a 41% MAE reduction (2524 -> 1470 MW). Hybridisation refines the deep-only result rather than replacing it.

XI. FUTURE WORK

- Train the residual model on out-of-sample (walk-forward) residuals to fix the distribution shift identified in Section VIII.
- Replace plain ARIMA with SARIMA(p,d,q)(P,D,Q,24) so that daily seasonality is absorbed by the linear component, leaving a smaller residual.
- Add exogenous regressors: temperature, calendar (holiday and weekend flags), day-type embeddings.
- Move from point forecasts to probabilistic outputs (quantile regression, DeepAR, or Temporal Fusion Transformer) to quantify forecast uncertainty.
- Online retraining on rolling windows to track distribution drift caused by heatwaves, post-pandemic demand shifts, or EV adoption.

XII. REFERENCES

[1] G. P. Zhang, "Time series forecasting using a hybrid ARIMA and neural network model," Neurocomputing, vol. 50, pp. 159-175, 2003.

[2] D. H. Wolpert, "Stacked generalization," Neural Networks, vol. 5, no. 2, pp. 241-259, 1992.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[4] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in Proc. ICLR, 2015.

[5] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, "Modeling long- and short-term temporal patterns with deep neural networks (LSTNet)," in Proc. SIGIR, 2018.

[6] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," Int. J. Forecast., 2021.

[7] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in Proc. ICLR, 2017.

[8] S. Ben Taieb, G. Bontempi, A. F. Atiya, and A. Sorjamaa, "A review and comparison of strategies for multi-step ahead time series forecasting," Expert Syst. Appl., vol. 39, no. 8, pp. 7067-7083, 2012.

[9] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control, Holden-Day, 1970.

[10] R. J. Hyndman and Y. Khandakar, "Automatic time series forecasting," J. Stat. Softw., 2008.